TECHNICAL HANDBOOK · HAND-OFF DOCUMENT

Security+ SY0-701 Lab Engine & AI Agent Governor

What was built, how every part works, and how to extend it.
Written for you, and for any AI you hand it to.

Repository	`delzarsolutionsllc/delzar-sovereign-wealth-machine`
Location in repo	`secplus_lab_engine/` (+ `.github/workflows/secplus-lab-engine.yml`)
Branch / Pull request	`ccr-2689f707-karox1` · PR #4 (draft, open, mergeable)
Commits	`6081f9b` labs 1–5 · `bb6d70d` challenge mode, CI, step-up MFA, labs 6–8
Size	~4,850 lines of Python · 8 labs · 8 sample scenario files · 1 policy file
Quality status	119 pytest tests passing · GitHub Actions green on Python 3.10, 3.11, 3.12, 3.13
Runtime requirements	Python 3.10+ standard library only. `rich` optional (colour); `pytest` for tests. Fully offline.
Document date	2026-11-03

Contents

1. How to use this document
2. Purpose & learning philosophy
3. Architecture & repository map
4. Command-line reference
5. Lab 1 — Joiner/Mover/Leaver & Access Governance
6. Lab 2 — Zero Trust (PE / PA / PEP)
7. Lab 3 — Log Forensics & Hash Verification
8. Lab 4 — Change Control & Impact Analysis
9. Lab 5 — AI Agent Guardrail Governor
10. Lab 6 — Risk Management & BIA
11. Lab 7 — Incident Response & Chain of Custody
12. Lab 8 — End-to-End SOC Pipeline
13. Challenge mode & progress tracking
14. Shared utilities
15. Policy configuration reference
16. Testing & CI
17. Design decisions & known limitations
18. How to extend (recipes + lab skeleton)
19. Roadmap: what to build next
20. Ready-to-paste prompt for another AI
21. Glossary

1. How to use this document

This handbook describes everything built so far: the purpose, the architecture, each lab's engine and decision rules, data formats, configuration, tests, and how to add more. Every function signature, file name and sample result was taken from the code as it stands, not written from memory.

> **If you are an AI reading this**
> The project is a modular Python CLI. Every lab follows the same **module contract** (Section 3.3). To extend it, follow that contract, keep to the standard library, load all policy from `config/default_policies.json` and scenarios from `data/*.json`, add pytest tests, and register the lab in `labs/__init__.py`. Section 18 has recipes and a full lab skeleton. Section 20 is a ready-made prompt.

> **If you are the learner**
> Read Sections 2–4 first, then one lab section at a time. Run that lab (`python main.py lab N`), then quiz yourself (`python main.py challenge N`) before reading its "Decision logic" table.

2. Purpose & learning philosophy

Goal: master the *manual* security operations behind CompTIA Security+ SY0-701 before designing *automated AI agents* that perform them. Each lab simulates one operational domain realistically enough to show:

1. **How the control works**: the actual decision logic, not just definitions.
2. **Where manual processes fail**: privilege creep, untested rollback plans, evidence destroyed by pulling the plug.

3. **How an AI agent should be governed** in that same domain: least privilege, human-in-the-loop, audit trails.

Every lab ends with an **"Exam & Architectural Insight" panel** with five parts: the SY0-701 objective, key concepts, manual failure modes, AI-governance guidance and exam tips.

The arc of the project: Labs 1–4 and 6–7 teach individual controls. Lab 5 shows how to put guardrails on automation. Lab 8 wires them all together into the architecture the learner is building toward: *detect → AI agent proposes → governor decides → human approves → the owning control executes*.

SY0-701 coverage

Domain	Objectives covered	Labs
1 General Security Concepts	1.2 Zero Trust · 1.3 Change management · 1.4 Cryptography (hashing, salting, HMAC)	2, 4, 3, 8
2 Threats, Vulnerabilities & Mitigations	2.4 Indicators of malicious activity	1, 3
3 Security Architecture	*Not yet covered* (see Roadmap)	—
4 Security Operations	4.6 IAM · 4.7 Automation & orchestration · 4.8 Incident response · 4.9 Investigation data sources	1, 2, 5, 7, 8, 3
5 Security Program Management	5.1 Governance (SoD, oversight) · 5.2 Risk management & BIA	1, 4, 5, 6

3. Architecture & repository map

3.1 Directory tree

```
secplus_lab_engine/
├── main.py                      CLI entrypoint: argparse subcommands + interactive menu
├── requirements.txt             rich (optional), pytest
├── README.md                    user-facing docs
├── .gitignore                   __pycache__, .pytest_cache, .venv, progress.json
├── config/
│   └── default_policies.json    ALL policy: RBAC/SoD, Zero Trust weights, threat intel,
│                                CMDB + maintenance windows, AI agent guardrails
├── data/                        scenario inputs (one per lab; editable)
│   ├── sample_identities.json      Lab 1  HR feed events + access checks
│   ├── sample_access_requests.json Lab 2  access requests, step-up outcomes, PEP attacks
│   ├── sample_logs.json            Lab 3  raw multi-source log stream + offline GeoIP
│   ├── sample_rfcs.json            Lab 4  six Requests for Change
│   ├── sample_agent_actions.json   Lab 5  20 agent proposals + human reviews
│   ├── sample_risks.json           Lab 6  five risk scenarios + BIA targets
│   ├── sample_incident.json        Lab 7  incident, action catalog, rookie/expert playbooks
│   └── sample_pipeline.json        Lab 8  directory, live sessions, simulated human decisions
├── labs/
│   ├── __init__.py              LABS registry: {number: (module, default data file)}
│   ├── lab1_jml_governance.py … lab8_soc_pipeline.py
├── utils/
│   ├── crypto_helpers.py        SHA-256, salt, PBKDF2, HMAC, canonical JSON, hash chains
│   ├── terminal_ui.py           UI class: rich or plain-text tables/panels/status lines
│   ├── loader.py                load_policies(), load_data(), load_json()
│   └── challenge.py             Question, run_challenge(), Progress (progress.json)
└── tests/
    ├── conftest.py              fixtures: policies, quiet_ui, data loader
    ├── test_labs.py             labs 1–5, crypto, demos, CLI
    └── test_extensions.py       challenge mode, step-up, labs 6–8, CLI challenge/progress
.github/workflows/secplus-lab-engine.yml   CI: pytest + smoke tests, Python 3.10–3.13
```

3.2 Design principles

- **Offline and standard-library only.** No network, no API keys. `rich` is optional; `UI(plain=True)` renders ASCII.
- **Policy as data.** Thresholds, roles, weights, inventories and guardrails live in `config/default_policies.json`; scenarios live in `data/`. Logic lives in Python. Changing JSON changes outcomes, and challenge answers follow automatically.
- **Pure engines, thin demos.** Each lab has testable engine classes and functions (no printing) plus a `run_demo()` that renders through the UI facade.
- **Deny by default, explain everything.** Every decision returns its reasons (strings), which feed the tables, challenge explanations and tests.
- **Tamper evidence.** Audit trails (log manifests, governor ledger, custody ledger) use salted SHA-256 hash chains and HMAC.
- **Deterministic.** Injectable clocks and seeds make results reproducible for tests and quizzes.

3.3 The lab module contract

Every `labs/labN_*.py` module exposes exactly these names, which `main.py` and the tests rely on:

Name	Type	Purpose
`TITLE`	str	Display title, e.g. "Lab 6 — Risk Management Calculator & BIA"
`OBJECTIVES`	str	SY0-701 objectives shown under the title
`INSIGHT`	dict	Keys: `objective` (str), `concepts`, `failure_modes`, `ai_governance`, `exam_tips` (lists of str)
`run_demo(ui, policies, data)`	→ dict	Runs the sample scenario, renders it, ends with `ui.insight(INSIGHT)`, returns a summary dict (used by tests)
`build_challenges(policies, data)`	→ list[Question]	Generates quiz questions *by running the engine* on the scenario

Registration in `labs/__init__.py`: `LABS = {N: (module, "sample_file.json"), …}`. The CLI, menu, "all", challenge mode and the parametrised tests pick up new labs automatically.

3.4 Runtime data flow

```
main.py ──load_policies()──▶ config/default_policies.json
   │      load_data(file) ──▶ data/sample_*.json   (or --file custom.json)
   ├─ lab N            → module.run_demo(ui, policies, data) → tables/panels → Insight panel → summary
   ├─ challenge N      → module.build_challenges(policies, data) → run_challenge() → progress.json
   └─ progress         → Progress().rows()  (weakest objective first)
```

4. Command-line reference

Run from inside `secplus_lab_engine/` (or call `main.py` by path; it fixes `sys.path` itself).

Command	What it does
`python main.py` / `menu`	Interactive menu: `1-8` run a lab · `C3` challenge lab 3 · `P` progress · `A` all · `T` tests · `Q` quit
`python main.py list`	Table of labs, objectives and default data files
`python main.py lab N [--file X.json]`	Run lab N's scenario (optionally your own data file)
`python main.py lab N --challenge`	Quiz mode for lab N
`python main.py challenge N|all [--seed S] [--file X.json]`	Quiz mode; `--seed` fixes question order
`python main.py progress`	Scores per objective (weakest first) + last 5 sessions
`python main.py all`	Run all eight labs back-to-back
`python main.py test [pytest args]`	Run the pytest suite
Global flags	`--plain` (no colour) · `--policies FILE` (alternate policy JSON) · `-v` (debug logging)

5. Lab 1 — Joiner/Mover/Leaver & Access Governance

4.6 IAM | 5.1 SoD | 2.4 privilege creep File: `labs/lab1_jml_governance.py` · Data: `sample_identities.json`

WHAT IT SIMULATES

A miniature Identity Governance & Administration (IGA) platform driven by an HR feed. Joiners get birthright access from their role. Movers should lose their old role's permissions; leaving them in place shows privilege creep. Leavers are disabled, fully revoked and get a deprovisioning ticket. Every access request passes a status gate, then RBAC, then ABAC.

ENGINE API

```
class IdentityGovernance(policy: dict, clock=None)
  joiner(record) -> Identity                # validates fields, role and clearance; provisions
role perms
  mover(user_id, new_role, new_department=None, revoke_previous=True) -> dict
                                            # revoke_previous=False simulates creep: old role
becomes an extra role
  grant_extra_role(user_id, role)           # ad-hoc "temporary" grant (creep source)
  leaver(user_id, reason) -> DeprovisionTicket
  suspend(user_id, reason)                  # IR use: disable but KEEP entitlements (used by Lab
8)
  check_rbac(user_id, permission) -> bool
  check_abac(user_id, resource, hour) -> (bool, reasons)
  evaluate_access(user_id, action, resource, hour=12) -> AccessDecision   # gate: STATUS | RBAC |
ABAC | ALLOW
  detect_sod(user_id=None) -> list[SoDConflict]
  access_review() -> list[dict]             # orphaned access, privilege creep
Dataclasses: Identity, AccessDecision, SoDConflict, DeprovisionTicket
```

DECISION LOGIC

Rule	Behaviour
Status gate	Only `allowed_employment_status` (default `active`) may access anything
RBAC	Permission must be in the union of the user's role permissions
ABAC: clearance	clearance level ≥ resource classification level (public 0 → secret 3)
ABAC: department	For `confidential` / `secret` data, user department must equal the data owner's
ABAC: time	`secret` data only inside business hours (07–19)
SoD	Flags a user holding both permissions of any `sod_conflicts` pair (e.g. `po:create` + `po:approve`)
Leaver SLA	Reason containing "cause" → priority P1, due in 1 hour, "cut access BEFORE notification"; otherwise P3, due in `leaver_sla_hours` (24)

DATA FORMAT (`SAMPLE_IDENTITIES.JSON`)

```
{ "events": [
    {"type":"joiner","identity":{"user_id","name","department","role","clearance","manager"}},
    {"type":"mover","user_id","new_role","new_department","revoke_previous":false,"note"},
    {"type":"grant","user_id","extra_role","note"},
    {"type":"leaver","user_id","reason","last_day"} ],
  "access_checks": [ {"user_id","action","resource":{"name","classification","department"},"hour"}
] }
```

SAMPLE OUTCOME

5 identities · **4 SoD conflicts** (SOD-01 PO create+approve, SOD-02 ghost vendor, SOD-03 code→prod, SOD-04 admin audits self) · 2 deprovisioning tickets (one P1) · 3 of 5 access checks denied · 3 privilege-creep review findings. Challenge mode: 12 questions (which SoD rule, ticket priority, ALLOW/DENY).

6. Lab 2 — Zero Trust Access Evaluator (PE / PA / PEP)

1.2 Zero Trust · 4.6 conditional access File: `labs/lab2_zero_trust_evaluator.py` · Data: `sample_access_requests.json`

WHAT IT SIMULATES

The NIST SP 800-207 model. In the **control plane**, the Policy Engine (PE) scores each request and the Policy Administrator (PA) issues or revokes short-lived signed tokens. In the **data plane**, the Policy Enforcement Point (PEP) checks every request before it reaches the resource.

ENGINE API

```
class PolicyEngine(policy).evaluate(req: AccessRequest) -> PolicyDecision(verdict, trust_score,
required_score, reasons)
class PolicyAdministrator(signing_key, ttl_seconds, clock)
  issue(req, decision, ttl=None) -> token | None      # only for ALLOW
  revoke(token) · revoke_subject(subject) -> count · verify(token) -> (ok, reason, claims)
  decode_unverified(token) -> claims                  # token = base64url(canonical JSON claims) +
"." + HMAC-SHA256
class PolicyEnforcementPoint(admin, resources).handle(token, resource, action="read") ->
EnforcementResult
class ZeroTrustGateway(policy, clock)                 # wires PE + PA + PEP
  request_access(req) -> (decision, token|None)
  complete_step_up(req, decision, mfa_passed) -> (final_decision, token|None)
Token claims: jti, sub, scope (one resource), act (one action), dev, iat, exp
```

TRUST ALGORITHM (SCORE STARTS AT 100)

Signal	Penalty	Signal	Penalty
IP reputation bad	−40 and **always DENY**	No MFA	−25
IP reputation suspicious	−15	User risk high (UEBA)	−30
Device health < 70	−30	Geo ≠ usual geo	−20
Device unmanaged	−20	Outside 07–19	−10

Checked first: an unknown resource → DENY; subject not entitled to the resource → DENY. Then: bad IP → DENY. If the resource requires MFA and none is presented → STEP_UP if score ≥ 50, else DENY. Score ≥ the resource's `min_score` → ALLOW; ≥ 50 → STEP_UP; otherwise DENY.

Step-up (adaptive identity): a failed MFA gives DENY and flags the subject as risky. A passed MFA re-evaluates with MFA present and adds 15. If that reaches the requirement, the result is ALLOW with a token TTL of 900/3 = **300 s**; otherwise DENY. MFA proves *who* is asking but cannot repair an unhealthy device.

PEP checks, in order: no token → malformed → HMAC signature → revoked → expired → scope ≠ resource → action mismatch.

SAMPLE OUTCOME

REQ-01 ALLOW · REQ-02 DENY (unmanaged BYOD, no MFA, 22:00) · REQ-03 STEP_UP then DENY (laptop health 65) · REQ-04 DENY (Alice's stolen creds from a Tor exit node) · REQ-05 STEP_UP then

ALLOW with a 300 s token · all **5 PEP attacks blocked** (expired, scope abuse, forged, revoked, no token). Challenge: 10 questions.

7. Lab 3 — Log Forensic Inspector & Hash Verifier

4.9 data sources | 2.4 indicators | 1.4 hashing File: `labs/lab3_log_inspector.py` · Data: `sample_logs.json`

PIPELINE

1. **Parse** raw lines `<ISO8601> <host> <source> key=value key="quoted value"` (shlex) into `LogEvent`. Sources: `FW`, `WinEvt`, `HTTP`, `EDR`. Malformed lines are counted as parse errors, not dropped silently.
2. **Enrich** with an offline GeoIP table (`geoip` in the data file).
3. **Detect** with correlation rules → `Finding(rule, severity, summary, evidence_lines, mitre)`, sorted CRITICAL → LOW.
4. **Extract IOCs** by regex: public vs RFC1918 IPs, SHA-256/SHA-1/MD5, user agents, domains.
5. **Verify integrity**: a salted SHA-256 hash chain over the lines plus an HMAC-signed manifest.

DETECTION RULES

Function	Rule	Logic	MITRE
`detect_brute_force`	BRUTE_FORCE(_SUCCESS)	≥5 × Event 4625 for one user+IP within 10 min; CRITICAL if a later 4624 comes from the same user+IP	T1110.001
`detect_password_spray`	PASSWORD_SPRAY	one IP with 4625s against ≥4 distinct accounts	T1110.003
`detect_impossible_travel`	IMPOSSIBLE_TRAVEL	consecutive 4624s for one user: distance >500 km and speed >900 km/h (haversine)	T1078
`detect_port_scan`	PORT_SCAN	one source → ≥10 distinct destination ports on FW	T1046
`detect_threat_intel`	TI_MATCH_IP / _USER_AGENT / _FILE_HASH	matches the bad IPs, UA patterns (sqlmap, nikto…) and hashes in config	—
`detect_sqli`	SQL_INJECTION	regex on HTTP `path` (`' OR 1=1`, `UNION SELECT`, …)	T1190
`detect_log_clearing`	AUDIT_LOG_CLEARED	Event ID 1102 / 104	T1070.001
`detect_time_gaps`	LOGGING_GAP	>30 min of silence on hosts in `continuous_log_hosts` (DC01, FW01)	T1562.002

To add a rule, write `detect_x(events, cfg) -> list[Finding]` and append it to `ALL_RULES`.

INTEGRITY MODEL

```
link[i]  = SHA256( salt || link[i-1] || canonical_json(line[i]) ),  link[-1] = "0"*64
manifest = {count, chain, head, signature = HMAC(manifest_hmac_key,
canonical_json({count,chain,head}))}
verify_manifest() -> VALID | TAMPERED (edit / delete / insert, with first broken index) |
MANIFEST_FORGED
```

SAMPLE OUTCOME

38 events, 1 malformed line, **14 findings (3 CRITICAL)**: jdoe brute-force success, EICAR malware hash, Security log cleared, password spray, Tulsa→Moscow impossible travel, port scan, SQLi and sqlmap/Nikto, TI IP matches, and a 50-minute DC01 logging gap. The integrity tests detect a deleted line, an edited IP, an injected line and a forged manifest. Challenge: 12 questions (identify the activity from raw lines; distractors exclude any rule that also fired on those same lines).

8. Lab 4 — Change Control & Impact Analyzer (CAB)

1.3 change management · 5.1 governance File: `labs/lab4_change_management.py` · Data: `sample_rfcs.json`

ENGINE API

```
class ChangeAdvisoryBoard(policy)
  dependents_of(system) -> set     # transitive "who depends on me" (blast radius)
  neighbours(system) -> set        # direct deps + direct dependents
  generate_backout_plan(rfc) -> list[str]   # reverse-order undo runbook via UNDO_MAP
  review(rfc, rehearse=True) -> ReviewResult(decision, risk_score, strategy, blockers, conditions,
                                             observations, blast_radius, generated_backout,
rehearsal)
rehearse_change(rfc, systems, fail_at_step=None) -> {failed_at_step, executed, restored, diffs}
```

REVIEW CHECKS

Check	Blocker (→ REJECT)	Condition
Schedule	outside Sat/Sun 00:00–06:00 (non-emergency); blackout date; end ≤ start	emergency outside window → ECAB + post-implementation review
Approvals	requester approved own change (SoD); fewer than 2 independent CAB approvals	system owner not among approvers
Testing	no staging test (non-emergency)	untested emergency
Impact	step effect `disable:X` breaks a neighbour tagged `requires:X`; a deny-list rule cuts off dependents; target not in CMDB	compensating control declared; allow-list additions
Backout	missing plan (non-emergency); reversible step with no undo; irreversible step with no prior backup; irreversible with no fail-forward plan; change + backout time exceeds window	missing plan (emergency) → auto-generated; no verification steps; fail-forward readiness
Rehearsal	executing the RFC's own backout does not restore the simulated state	—

Decision: any blocker → **REJECT**; risk ≤ 3 with no conditions → **APPROVE**; otherwise **APPROVE_WITH_CONDITIONS**. Strategy: ROLLBACK, FAIL_FORWARD (irreversible step plus hotfix plan), or NONE.

RFC FORMAT

```
{id, title, requested_by, approvals[], target_systems[], window{start,end}, estimated_minutes,
 tested_in_staging, emergency, compensating_controls{system: text}?,
 change_steps[{step, action, target, patch?|rule?|setting?, effects?["disable:tls1.0"]}],
 firewall_changes{allow[], deny["port:1521"]},
 backout_plan{estimated_minutes, steps[{undo_of, action}], verification[]} | null,
 fail_forward_plan{hotfix_steps[], on_call}?}
actions: snapshot backup verify notify health_check | apply_patch config_change firewall_deny
firewall_allow
         stop_service start_service restart_service | irreversible: db_schema_migrate
firmware_flash data_purge
```

SAMPLE OUTCOME

RFC	Decision	Why
0420 web-frontend patch	APPROVE	in window, quorum, tested, backout rehearsal PASS
0412 OpenSSL + disable TLS 1.0	REJECT	breaks legacy `erp-legacy`, which requires TLS 1.0
0415 block port 1521	REJECT	6 blockers: Wednesday 14:00, self-approval, no quorum, untested, cuts off Oracle dependents
0423 payroll schema migration	APPROVE_WITH_CONDITIONS	irreversible → FAIL_FORWARD with on-call hotfix
0430 EMERGENCY xz patch	APPROVE_WITH_CONDITIONS	ECAB, untested, auto-generated backout
0431 disable SMBv1	REJECT	Black Friday blackout; backout rehearsal FAILS (restarts instead of restoring config)

9. Lab 5 — AI Agent Supervisor & Guardrail Governor

4.7 automation | 4.6 non-human PAM | 5.1 HITL File: `labs/lab5_ai_agent_governor.py` · Data: `sample_agent_actions.json`

MODEL

Agents never act on production directly. Every proposed `AgentAction(id, agent_id, action, target, parameters, justification, cab_ticket, confidence, t)` goes to the `AgentGovernor`, which returns **ALLOW**, **REQUIRE_HUMAN_APPROVAL** (HITL) or **BLOCK**. The strictest verdict among all violations wins.

```
class AgentGovernor(policy)
  evaluate(action) -> GovernorDecision(action_id, agent_id, verdict, violations[], final, reviewer)
  human_review(action_id, reviewer, approve, reason) -> GovernorDecision
       # only HITL items; reviewer must be in human_approvers; agents can't approve; BLOCK is never
overridable
  release_quarantine(agent_id, reviewer)
  ledger_chain(salt) / verify_ledger(chain, salt)      # hash-chained decision ledger
```

CHECKS (IN ORDER)

Code	Verdict	Trigger
AGENT_QUARANTINED / UNKNOWN_AGENT	BLOCK	kill switch engaged / unregistered identity ("shadow AI")
OUT_OF_SCOPE_ACTION	BLOCK	action not in the agent's `allowed_actions`
HALLUCINATED_ASSET / _PATCH / _TICKET	BLOCK	target not in `asset_inventory` (for apply_patch, restart_service, isolate_host, run_command); patch not in catalog; CAB ticket not approved
IMMUTABLE_RULE	BLOCK	firewall rule id in `immutable_firewall_rules` ([0])
OVERLY_PERMISSIVE_RULE	HITL	src any + port any
SELF_ESCALATION / PRIV_GRANT_NO_CAB	BLOCK	granting to an agent identity / privileged role without an approved CAB ticket
PROTECTED_CONTROL / AUDIT_TAMPERING	BLOCK	touching mfa, edr, audit_logging, siem_forwarder, backup_job / deleting or modifying logs
DANGEROUS_COMMAND	BLOCK	regexes: `rm -rf /`, `curl|sh`, `chmod 777`, disabling Defender, ...
PROMPT_INJECTION	BLOCK	"ignore previous instructions", "developer mode", ... in the justification
RATE_LIMIT	BLOCK	more than 5 actions per agent in 60 s
LOW_CONFIDENCE / CRITICAL_ASSET / HUMAN_REQUIRED_CLASS / NO_JUSTIFICATION	HITL	confidence < 0.80 / disruptive action on a tier-0 asset / action in `always_hitl_actions` (disable_account) / empty rationale

Quarantine: 3 BLOCKs puts the agent in quarantine (kill switch); only a human approver can release it.

SAMPLE OUTCOME

20 actions → 8 ALLOW · 2 HITL · **10 BLOCK**. patch-bot-01 is quarantined after an invented server and patch, a `curl | bash`, and a restart storm. Human review: A09 approved; A04 rejected; an override of rule 0 refused; an agent approving itself refused; an unauthorised user refused. Editing the ledger after the fact is detected. Challenge: 20 questions.

10. Lab 6 — Risk Management Calculator & BIA

5.2 risk management File: `labs/lab6_risk_management.py` · Data: `sample_risks.json`

FORMULAS & API

```
sle(AV, EF) = AV × EF          ale(SLE, ARO) = SLE × ARO        availability(MTBF, MTTR) = MTBF /
(MTBF + MTTR)
assess(risk, appetite_ale) -> RiskAssessment(sle, ale, likelihood, impact, options[], treatment,
rationale, residual_ale)
check_bia(entry) -> BIAResult(rpo_ok, rto_ok, availability, availability_ok, issues)
band(value, bands) -> 1..5     ARO_BANDS ≤0.05/0.2/1/5 ; SLE_BANDS ≤$10k/$100k/$500k/$2M
```

TREATMENT LOGIC

1. ALE ≤ risk appetite ($10,000) → **ACCEPT** (the risk owner signs).
2. Otherwise compute net annual benefit: MITIGATE = ALE − ALE_after − control cost; TRANSFER = ALE × coverage − premium; AVOID = ALE − cost of stopping (only if `activity_optional`).
3. Choose the best positive option. If none is positive → **EXCEPTION** (leadership sign-off with an expiry date).

SAMPLE OUTCOME (TOTAL ALE $631,000 → RESIDUAL $58,800)

Risk	ALE	Treatment
R-01 Customer DB breach (AV $2M, EF 40%, ARO 0.25)	$200,000	MITIGATE (WAF + encryption)
R-02 Laptop theft (AV $3k, EF 100%, ARO 12)	$36,000	MITIGATE (FDE + MDM)
R-03 Data-center flood (AV $5M, EF 60%, ARO 0.01)	$30,000	TRANSFER (insurance beats $400k relocation)
R-04 Microsite defacement	$5,000	ACCEPT (within appetite)
R-05 Legacy anonymous FTP	$360,000	AVOID (retire FTP → SFTP portal)

BIA: payroll-app **fails RPO** (24 h backups vs 4 h RPO); erp-legacy **fails RTO** (12 h restore vs 4 h). The demo also prints a 5×5 likelihood × impact heat map. Challenge: 14 questions (compute ALE, choose treatment, BIA verdict).

11. Lab 7 — Incident Response Runner & Chain of Custody

4.8 incident response 4.9 File: `labs/lab7_incident_response.py` · Data: `sample_incident.json`

MODEL

A playbook is an ordered list of action ids from a catalog. Each action has a `phase` (preparation, detection, analysis, containment, eradication, recovery, lessons_learned) and may have `evidence{source, host, volatility_rank 1–6, hashed}`, `destroys_up_to_rank`, `requires[]` and `anti_pattern`. `grade_playbook(name, playbook, catalog, incident_id)` returns `PlaybookGrade(score, grade, issues, custody, ledger, ledger_chain)`.

Issue code	Penalty	Trigger
EVIDENCE_DESTROYED	−20	power-off/reimage destroys uncollected evidence; or collecting evidence already destroyed
CHAIN_OF_CUSTODY	−15	evidence acquired without a hash
MISSING_PREREQ	−15	e.g. reimage before `image_disk` and `legal_hold`
NO_CONTAINMENT	−15	no containment step at all
ORDER_OF_VOLATILITY	−10	more volatile evidence collected after less volatile (RFC 3227)
PHASE_ORDER	−10	eradication/recovery before containment; recovery before eradication; containment after recovery; anything after lessons learned
NO_LESSONS_LEARNED / ANTI_PATTERN / UNKNOWN_ACTION	−10 / −10 / −5	missing review / flagged bad practice / not in catalog

Grade: A ≥ 90, B ≥ 80, C ≥ 70, D ≥ 60, otherwise F. Volatility ranks: 1 registers/cache, 2 RAM/process/network state, 3 swap/temp, 4 disk, 5 remote logs, 6 archives/backups.

SAMPLE OUTCOME

Rookie: 0/100 (F): pulled the plug, copied files to a personal USB unhashed, reimaged with no disk image or legal hold, tried to capture RAM after power-off, disabled the account after recovery, no lessons learned. **Expert: 100/100 (A)**: 5 evidence items hashed in volatility order and a 10-entry custody ledger; editing who collected EV-001 breaks the chain. Challenge: 16 questions (phase identification, what to collect first, grade the playbook).

12. Lab 8 — End-to-End SOC Pipeline

4.7 SOAR · 4.8 · 1.3 · 4.6 · 1.2 File: `labs/lab8_soc_pipeline.py` · Data: `sample_pipeline.json` (+ `sample_logs.json`, `sample_incident.json`)

```
Lab 3 Detect ──▶ AI SOC agent proposes ──▶ Lab 5 Governor ──▶ human (HITL)
                                                  │
          disable_account              block_ip                    isolate_host
   Lab 1 suspend + Lab 2          Lab 4 emergency RFC → CAB    Lab 7 capture volatile evidence,
   PA.revoke_subject()            review + rehearsed backout   hash it, THEN isolate
```

API

```
soc_agent_propose(findings, events_by_line, agent_id) -> list[Proposal]   # deterministic stand-in
for an LLM
run_pipeline(policies, data, logs, incident) -> PipelineResult(rows, rfc_decision, suspended,
                                                               sessions_revoked, isolated,
custody_records)
```

Agent mapping: BRUTE_FORCE_SUCCESS → disable_account + block_ip · IMPOSSIBLE_TRAVEL → disable_account (confidence 0.71) · TI_MATCH_FILE_HASH → isolate_host · PASSWORD_SPRAY / SQL_INJECTION / PORT_SCAN → block_ip (deduplicated) · AUDIT_LOG_CLEARED → **overreach**: modify firewall rule 0. Proposals are spaced 15 s apart (the agent paces itself). All approved IP blocks are batched into one emergency RFC (`RFC-AUTO-0001`, requested by the agent, approved by the emergency CAB).

SAMPLE OUTCOME

Finding	Proposal	Governor	Human	Outcome
Brute-force success	disable jdoe	HITL	approve	suspended; 2 sessions revoked
Brute-force success	block 45.155.205.233	ALLOW	—	RFC → APPROVE_WITH_CONDITIONS, rehearsal PASS
Malware hash	isolate ws-0142	ALLOW	—	2 volatile evidence items hashed before isolation
Log cleared	modify rule 0	BLOCK	—	OUT_OF_SCOPE_ACTION; IMMUTABLE_RULE
Password spray / SQLi	block 2 IPs	ALLOW	—	same RFC
Impossible travel	disable asmith	HITL	reject	false positive (travel VPN) → tune the rule

Challenge: 8 questions ("where does this proposal end up?", "what must happen before isolation?").

13. Challenge mode & progress tracking

File: `utils/challenge.py`. Questions are **computed by running each engine**, so they always match the current policy and data. 99 questions in total across the 8 labs.

```
@dataclass Question(prompt, choices, answer, explanation, objective, context="")   # answer must be
in choices
run_challenge(ui, lab_number, questions, ask=None, progress=None, shuffle=True, seed=None) ->
ChallengeResult
    # accepts option number or exact text; shows PASS/FAIL + "Why"; EOF/Ctrl-C ends early, scoring
answered only
class Progress(path=progress.json): record(objective, correct) · record_session(result) · save() ·
rows()
    # rows sorted weakest first; status PASS ≥85%, WARN ≥65%, FAIL below (LOW/MEDIUM if <3
attempts)
```

A test answers every lab's questions with the engine's own answers and requires 100%. That catches unanswerable or ambiguous questions.

14. Shared utilities

Module	Contents
`crypto_helpers.py`	`sha256_hex`, `generate_salt` (128-bit), `salted_sha256`, `stretch_password` (PBKDF2-HMAC-SHA256, 100k iterations), `file_sha256`, `constant_time_equals`, `hmac_sign` / `hmac_verify`, `canonical_json`, `build_hash_chain` / `verify_hash_chain` (returns valid, first_broken_index, length_delta)
`terminal_ui.py`	`UI(plain=False, quiet=False)`: `header`, `section`, `status(label, verdict, detail)` (verdict colour from `STATUS_STYLES`), `bullet`, `table`, `panel`, `insight(INSIGHT)`, `ask`. Falls back to ASCII when rich is missing.
`loader.py`	`ROOT`, `CONFIG_DIR`, `DATA_DIR`, `load_json`, `load_policies(path=None)`, `load_data(name)`

15. Policy configuration reference

File: `config/default_policies.json`. Swap the whole file with `--policies`.

Section	Keys
`identity_governance`	clearance_levels · roles{role: [perms]} (9 roles) · sod_conflicts[{id,a,b,risk}] (5) · abac_rules{require_clearance_gte_classification, require_department_match_for, business_hours, restrict_outside_hours_for, allowed_employment_status} · leaver_sla_hours
`zero_trust`	token_ttl_seconds (900) · step_up{bonus 15, ttl_divisor 3} · signing_key · thresholds{allow 70, step_up 50} · weights{…} · min_device_health 70 · business_hours · resource_sensitivity{res: {min_score, requires_mfa}} · entitlements{subject: [resources]}

`log_inspection`	Event IDs (4625, 4624, 1102/104, 4672) · brute_force_threshold 5 / window 10 min · impossible_travel_kmh 900 · port_scan_distinct_ports 10 · continuous_log_hosts · max_log_gap_minutes 30 · threat_intel{bad_ips, bad_user_agent_patterns, bad_file_hashes} · integrity_salt · manifest_hmac_key
`change_management`	maintenance_windows · blackout_dates · cab_quorum 2 · cab_members · max_risk_for_auto_approve 3 · systems{name: {owner, legacy, depends_on, tags}} (the CMDB, 9 systems incl. edge-fw) · irreversible_actions
`ai_governance`	agents{id: {type, allowed_actions}} (4) · immutable_firewall_rules · always_hitl_actions · privileged_roles · protected_controls · confidence_threshold 0.80 · max_actions_per_agent_per_window 5 · rate_window_seconds 60 · quarantine_after_blocks 3 · human_approvers · dangerous_command_patterns · prompt_injection_patterns · asset_inventory · patch_catalog · approved_cab_tickets

> **Note**
> All keys, salts and data are fictional placeholders for the lab only. This is a training simulator, not a production control.

16. Testing & CI

- **119 tests**: `tests/test_labs.py` (crypto vectors, Labs 1–5 engines, demo headline numbers, CLI) and `tests/test_extensions.py` (challenge framework, step-up, session revocation, suspend, HITL class, key separation, Labs 6–8, CLI challenge/progress).
- Parametrised tests run every lab in `LABS`, so a new lab is smoke-tested automatically.
- **CI** (`.github/workflows/secplus-lab-engine.yml`): triggers on changes under `secplus_lab_engine/**`; matrix Python 3.10–3.13; installs requirements, runs pytest, runs `main.py all` with rich, then uninstalls rich and runs `--plain all`. All jobs currently green.

17. Design decisions & known limitations

Decisions worth knowing

- **The "AI agent" in Lab 8 is deterministic** (`soc_agent_propose`), so results are reproducible and testable. It is designed to be swapped for a real model; every guardrail still applies.
- **Agent pacing:** Lab 8 spaces proposals 15 s apart. At 5 s, Lab 5's rate limiter blocked legitimate incident response. Lab 5 still demonstrates a runaway agent.
- **disable_account always requires a human** (`always_hitl_actions`), however confident the agent is.
- **Rate limiting counts attempted actions, including blocked ones.** This is a defensible choice; you could argue for counting only executed actions.
- **BLOCK is never overridable** through `human_review`: a human must change policy, not approve an exception at runtime.
- **Key separation:** the evidence-sealing key is separate from the Zero Trust token key.

Known limitations

- Simulated environments only: no real AD, firewall, SIEM or EDR integration.
- The log parser expects the lab's `key=value` format, not real EVTX/syslog/CEF.
- The Lab 4 rehearsal models only a few state attributes (patches, config version, running, firewall lists, schema).
- The sample scenarios are fixed. Challenge mode re-asks the same questions in a different order, so you can memorise the answers (a random scenario generator is on the roadmap).
- CI runs twice per PR push (push + pull_request triggers). Harmless, and can be trimmed.
- SY0-701 Domain 3 (architecture) and objectives 2.2, 2.3, 4.1–4.5, 5.3–5.6 are not yet covered.

18. How to extend (recipes)

18.1 Add a new lab

1. Create `labs/labN_topic.py` following the contract (skeleton below).
2. Put thresholds in a new section of `config/default_policies.json`; put the scenario in `data/sample_topic.json`.
3. Register it in `labs/__init__.py`: import it and add `N: (labN_topic, "sample_topic.json")`.
4. Add engine tests in `tests/`. The parametrised demo and challenge tests cover it automatically: they need ≥5 valid questions and 100% on self-answer.
5. Update `README.md` (labs table, data files, practice ideas).

```
"""Lab N — Title.  Security+ SY0-701 objectives: X.Y ...  Run: python main.py lab N"""
from __future__ import annotations
from dataclasses import dataclass, field

TITLE = "Lab N — Title"
OBJECTIVES = "SY0-701 X.Y Topic"

@dataclass
class Result:
    item_id: str
    verdict: str                        # use words in terminal_ui.STATUS_STYLES for colour
    reasons: list[str] = field(default_factory=list)

def evaluate(item: dict, cfg: dict) -> Result:
    """Pure decision logic: no printing, deterministic, returns reasons."""
    reasons = []
    if item["value"] > cfg["threshold"]:
        reasons.append(f"value {item['value']} > threshold {cfg['threshold']}")
    return Result(item["id"], "FAIL" if reasons else "PASS", reasons or ["within policy"])

INSIGHT = {"objective": "SY0-701 X.Y ...", "concepts": [...], "failure_modes": [...],
           "ai_governance": [...], "exam_tips": [...]}

def run_demo(ui, policies: dict, data: dict) -> dict:
    cfg = policies["topic"]
    results = [evaluate(i, cfg) for i in data["items"]]
    ui.section("Step 1 — Evaluate")
    ui.table("Results", ["ID", "Verdict", "Reasons"], [[r.item_id, r.verdict, "; ".join(r.reasons)]
for r in results])
    ui.insight(INSIGHT)
    return {"fail": sum(r.verdict == "FAIL" for r in results)}

def build_challenges(policies: dict, data: dict) -> list:
    from utils.challenge import Question
    cfg = policies["topic"]
    return [Question(prompt=f"{i['id']}: PASS or FAIL?", context=str(i), choices=["PASS", "FAIL"],
                     answer=evaluate(i, cfg).verdict, explanation="; ".join(evaluate(i,
cfg).reasons),
                     objective="X.Y") for i in data["items"]]
```

18.2 Smaller extensions

Goal	Where
New log detection rule	`detect_x(events, cfg) -> list[Finding]` in Lab 3, append to `ALL_RULES`, add to `FRIENDLY` for challenges
New agent guardrail	Add a check inside `AgentGovernor._check` using `add(code, BLOCK|HITL, msg)`; add config keys under `ai_governance`
New agent or permission	`ai_governance.agents` in config
New CMDB system / legacy dependency	`change_management.systems` with `tags` like `requires:tls1.0` or `port:1521`
New change action	`UNDO_MAP` and `_apply()` in Lab 4 (and `irreversible_actions` if it can't be undone)
New IR action	`actions` catalog in `sample_incident.json` (phase, evidence, destroys_up_to_rank, requires, anti_pattern)
Plug in a real LLM agent	Replace `soc_agent_propose()` in Lab 8 with a model call that returns `AgentAction`s; keep it opt-in so the core stays offline

19. Roadmap: what to build next

#	Idea	Objectives	Notes
1	Random scenario generator	all	New logs/RFCs/agent actions each run so challenge answers can't be memorised; seedable
2	Vulnerability triage lab	4.3, 2.3	CVSS × asset criticality × known-exploited status; SLAs; exceptions; compensating controls
3	Email security analyzer	4.5, 2.2	Parse raw headers: SPF/DKIM/DMARC alignment, spoofing, phishing indicators
4	Firewall rule auditor	3.2, 4.5	Shadowed/redundant rules, any/any, missing implicit deny, segmentation
5	PKI & certificate lab	1.4	Chain validation, expiry, CRL/OCSP, SAN/wildcard scope, key escrow
6	Secure baseline / hardening checker	4.1	Compare host configs against a CIS-style baseline; drift detection
7	Third-party / vendor risk	5.3	Questionnaire scoring, SLA/MSA/BPA checks, right-to-audit
8	Optional real-LLM mode	4.7	Model-written proposals in Lab 5/8 to test the guardrails against real hallucinations and injection
9	Exam simulator	all	Timed, mixed-domain run of 90 questions drawn from every lab, scored like the real exam

20. Ready-to-paste prompt for another AI

Attach this PDF (and ideally the `secplus_lab_engine/` folder), then paste:

```
You are extending an existing offline Python 3.10+ project called "secplus_lab_engine": a
Security+ SY0-701 lab simulator with 8 labs, challenge mode and an AI-agent guardrail governor.
The attached handbook describes its architecture, module contract, APIs, data formats and tests.

Rules:
1. Follow the lab module contract exactly: TITLE, OBJECTIVES, INSIGHT (objective, concepts,
   failure_modes, ai_governance, exam_tips), run_demo(ui, policies, data) -> dict,
   build_challenges(policies, data) -> list[Question].
2. Standard library only (rich optional through utils/terminal_ui.UI). No network calls.
3. Policy/thresholds go in config/default_policies.json (new section); scenarios go in
   data/sample_*.json. Engines are pure (no printing) and return reasons for every decision.
4. Register the lab in labs/__init__.py LABS = {N: (module, "sample_file.json")}.
5. Write pytest tests for the engine logic; the existing parametrised tests require
   >= 5 challenge questions, unique choices, and 100% when answered with the engine's own answers.
6. Make challenge distractors genuinely wrong (no second correct answer).
7. End every demo with ui.insight(INSIGHT), including how an AI agent should be governed.

Task: build Lab 9 — Vulnerability Triage (SY0-701 4.3 / 2.3). Prioritise findings by CVSS,
asset criticality (from the change_management.systems CMDB), known-exploited status and exposure;
assign remediation SLAs; handle exceptions with compensating controls and expiry dates; and
generate a challenge set. Show me each new/changed file in full, then the tests.
```

Swap the "Task" paragraph for any roadmap item, e.g. "build the random scenario generator" or "add an optional LLM-backed soc_agent_propose behind a --llm flag".

21. Glossary

ABAC	Attribute-based access control
ALE / SLE / ARO / EF / AV	Annualised loss expectancy / single loss expectancy / annualised rate of occurrence / exposure factor / asset value
BIA	Business impact analysis (RTO, RPO, MTTR, MTBF)
CAB / ECAB	(Emergency) Change Advisory Board
CMDB	Configuration management database (systems + dependencies)
HITL	Human-in-the-loop (REQUIRE_HUMAN_APPROVAL)
HMAC	Keyed hash: integrity + authenticity
IGA	Identity governance & administration

IOC	Indicator of compromise
JML	Joiner / Mover / Leaver lifecycle
PE / PA / PEP	Policy Engine / Policy Administrator / Policy Enforcement Point
RBAC	Role-based access control
RFC	Request for Change (Lab 4) · RFC 3227 = evidence collection guideline (Lab 7)
RPO / RTO	Max tolerable data loss / downtime
SoD	Separation of duties
SOAR	Security orchestration, automation & response